

Control of Servo Motors Utilizing Multi-layer Neural Networks and AC Motor

Ali Elbadawy

49-33967

Salma Essam

49-9180

AC Servo Motor

AC Servo Motors are widely used for the precise control of dynamic loads. Many control techniques consisting of advanced hardware and Sensors were used to achieve the objectives. However, many internal and external factors have an impact on the precision of control for the AC Servo Motor. The precise control of position, speed, and torque were the main problems on AC Servo Motor. Such Motors were extremely demanded in the various control applications, i.e., Process/industrial control, medical equipment, Computer Numerical Control machines, conveying, robotics, textiles, Liquid Cristal Display manufacturing and etc. [1]

AC servomotor is basically a two phase reversible induction motor and is capable of providing the desired response characteristics, due to their smaller diameter and high resistance rotors. The small diameter provides low inertia and helps for faster starts, stops and speed reversals while high resistance helps to shape the speed torque characteristics suitable for accurate control.[5]

Modelling of AC Servo Motor (Position Control)

0.1 Control torque equation:

$$T_c = K_v \cdot \omega + K_i \cdot \dot{\omega}$$

T_c : Control torque (Nm)

$\vec{K}_v, \vec{K}_i$: Motor constants (Nm/V, Nm/rad/s)

ω : Angular velocity of the AC servo motor (rad/s)

V_{rated} : Rated input voltage (V)

0.2 Dynamic equation of the mechanical system

$$T_c = J \cdot \ddot{\theta} + B \cdot \dot{\theta}$$

$\dot{\theta}$: Angular position of the AC servo motor (rad)

$\ddot{\theta}$: Angular acceleration of the AC servo motor (rad/s²)

B : Friction coefficient

J : Moment of inertia (Kg.cm²)

0.3 Equation obtained by equating the two equations

$$J \cdot \ddot{\theta} + B \cdot \dot{\theta} = K_v \cdot \omega + K_i \cdot \dot{\omega}$$

0.4 Transfer function between $\theta(s)$ and $V(s)$

$$\frac{\theta(s)}{V(s)} = \frac{Km}{s(\tau \cdot s + 1)} \quad [2]$$

$$K : \text{Motor gain constant} = \frac{K1}{K2 + B}$$

$$\tau : \text{Motor time constant} = \frac{J}{K2 + B}$$

Modelling of AC Servo Motor (Speed Control)

0.5 Direct Torque Control

$$\begin{bmatrix} V_{qs} \\ V_{ds} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} V_{as} \\ V_{bs} \end{bmatrix} \quad (1)$$

$$\begin{bmatrix} V_{qr} \\ V_{dr} \end{bmatrix} = \begin{bmatrix} \cos \theta_r & -\sin \theta_r \\ -\sin \theta_r & -\cos \theta_r \end{bmatrix} \begin{bmatrix} V_{ar} \\ V_{br} \end{bmatrix} \quad (2)$$

$$\frac{d\psi_{qs}}{dt} = \omega_b(v_{qs} + \frac{r_s}{x_{ls}}(\psi_{mq} - \psi_{qs})) \quad (3)$$

$$\frac{d\psi_{ds}}{dt} = \omega_b(v_{qr} + \frac{r_s}{x_{ls}}(\psi_{md} - \psi_{ds})) \quad (4)$$

$$\frac{d\psi_{qr}}{dt} = \omega_b(v_{qr} - \frac{\omega_r}{\omega_b}\psi_{dr} + \frac{r_r}{x_{lr}}(\psi_{mq} - \psi_{qr})) \quad (5)$$

$$\frac{d\psi_{dr}}{dt} = \omega_b(v_{dr} - \frac{\omega_r}{\omega_b}\psi_{qr} + \frac{r_r}{x_{lr}}(\psi_{md} - \psi_{dr})) \quad (6)$$

$$\psi_{mq} = x_m(i_{qs} + i_{qr}) \quad (7)$$

$$\psi_{md} = x_m(i_{ds} + i_{dr}) \quad (8)$$

$$T_{em} = \frac{3}{2} \frac{P}{2} \omega_b(\psi_{ds}i_{qs} - \psi_{qs}i_{ds}) \quad (9)$$

$$J \frac{d\omega_r}{dt} = T_{em} + T_{mec} - T_{dam} \quad (10)$$

$$T_{em} = \frac{3P}{2} \omega_b(\psi_s i_s q_s - \psi_s q_s i_s) \quad (9)$$

The dynamics of the rotor is given by equation

$$J \frac{d\omega_{rm}}{dt} = T_{em} + T_{mec} - T_{damp} \quad (10)$$

Control Theory : Neural-based PID controller

The predominant control method utilized in servo motor systems is the Proportion-Integration-Differentiation (PID) control strategy, which derives control actions solely from the tracking error without relying on the specific mathematical model of the controlled object [2]. Despite its widespread use, setting PID parameters usually involves manual adjustment, depending heavily on the expertise of engineers. Additionally, changes in motor conditions such as magnetic saturation, temperature, and load during operation can diminish the effectiveness of the initially set PID parameters. Consequently, manual tuning becomes challenging and time-consuming, requiring frequent adjustments to maintain optimal performance. To address these issues and enhance control effectiveness, an adaptive control algorithm is required in servo systems. This algorithm must be capable of dynamically adjusting control parameters based on real-time identification of relevant parameter values to accommodate changing working conditions of the servo motor.

Implementing neural networks in adaptive PID control is one technique used to regulate the speed of a servo motor amid changing system parameters. The introduction of adaptive PID control aimed to overcome the constraints associated with fixed-gain PID controllers. This capability is facilitated by the ability of the proposed controller to finely adjust PID parameters with increased precision using neural network technology. Remarkably, this adjustment process relies solely on input-output pairs, even for control objects with unknown Jacobians.

An approach to implementing this concept involves utilizing a feed-forward neural network architecture[5]. This neural network is fed with inputs representing variations in motor parameters and dynamic load conditions, and it generates PID controller gains as outputs. Initially, a diverse input matrix containing various parameter values is supplied to the neural network, which undergoes training using the gradient descent back-propagation algorithm. This algorithm computes derivatives of performance relative to weight and bias variables, adjusting each variable iteratively according to the gradient descent equation:

$$\theta_j = \theta_j - \alpha \frac{\partial J(\theta_0, \theta_1, \dots, \theta_n)}{\partial \theta_j} \quad (13)$$

Furthermore, to fine-tune the proportional, integral, and derivative gains of the PID controller based on motor parameter variations, three distinct single-output, multiple-input neural networks are designed. Each neural network is trained using the same set of training parameters employing the gradient descent back-propagation algorithm. The

rationale behind employing three independent neural networks lies in the superior minimization of the mean square error parameter as a performance metric. This approach outperforms using a single network with multiple outputs.

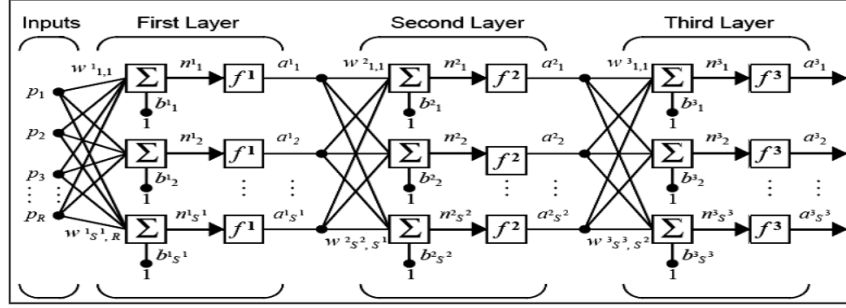


Figure 1: Multi-layer neural network architecture

A recent emerging approach involves integrating Recurrent Neural Networks (RNNs) with PID control [3]. RNNs are networks featuring both feedforward and internal feedback connections, incorporating unit delays. They consist of layers of nodes connected via feedforward links. The distinctive feature of RNNs lies in their internal states, where input along a recurrent connection depends on the previous output of the processing element, thereby capturing the entire past history of the processing element.

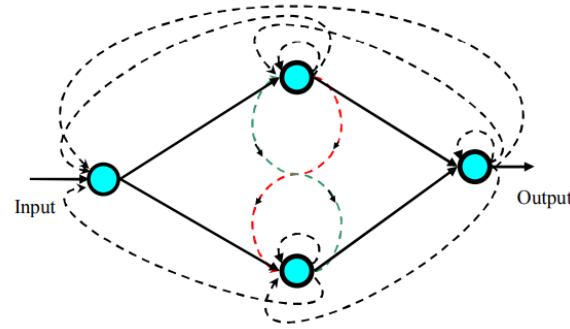


Figure 2: Fully Connected Recurrent Neural Network (RNN)

In this adaptive control scheme, two neural networks are employed: one acts as the PID controller, while the other serves as the plant emulator[4]. The initial step involves developing the architectures of the PID controller and emulator networks. Subsequently, training algorithms are formulated based on the Real-Time Recurrent Learning (RTRL) methodology and classical Back-propagation technique.

Converters

0.6 Rectifier

The rectifier converts the incoming AC power from the mains supply into DC power. This is typically achieved using diodes arranged in a bridge configuration. The rectification process converts the alternating current (AC) waveform into a pulsating direct current (DC) waveform. DC Bus: The DC bus is a capacitor bank that smoothens the pulsating DC voltage output from the rectifier, providing a stable DC voltage source for the inverter section. The DC bus helps in reducing voltage fluctuations and ensuring a consistent DC voltage level. Front-end Converter (Optional): In some designs, especially those requiring power factor correction or regeneration capabilities, a front-end converter may be included. This converter performs additional power conditioning functions such as power factor correction (PFC) to improve the overall power quality of the system.

0.7 Inverter

IGBTs or Power Semiconductors: The inverter section utilizes insulated gate bipolar transistors (IGBTs) or other power semiconductor devices to generate the variable frequency and voltage output required to drive the AC motor. These devices switch the DC voltage from the DC bus on and off rapidly, creating a variable voltage and frequency output. Pulse Width Modulation (PWM): The inverter section typically employs a pulse width modulation (PWM) technique to control the output voltage and frequency. PWM involves varying the width of the pulses of the switching waveform while keeping the frequency constant. By adjusting the pulse width, the effective voltage and frequency applied to the motor can be controlled, thereby regulating the motor speed.

Simulation

0.8 Direct Torque Controller

The Direct Torque Controller (DTC) is a control scheme widely used for induction motors. Its basic configuration consists of a hysteresis controller, torque and flux estimators, and a switching table, as depicted in Figure 3. Unlike vector control systems, DTC is characterized by its simplicity, lacking coordinate transformations, PI regulators, pulse-width modulators, and position encoders. This simplicity translates to reduced delays and mechanical requirements.

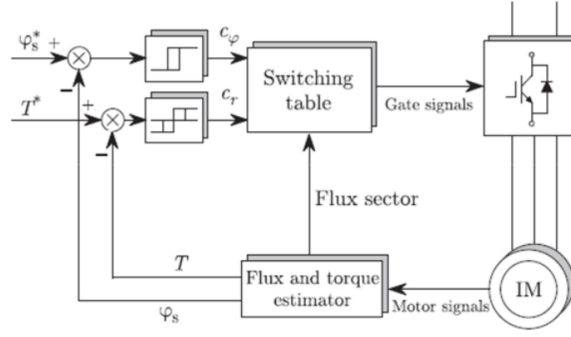


Figure 3: Direct Torque Control of Induction Machine

The fundamental principle behind DTC is to directly control the stator (or rotor) flux linkage and electromagnetic torque by selecting optimal inverter switching modes. This approach offers fast response and low harmonic losses by constraining flux and torque errors within hysteresis bands. The DTC controller integrates hysteresis controllers for flux and torque, ensuring the selection of appropriate voltage vectors to maintain desired flux and torque levels.

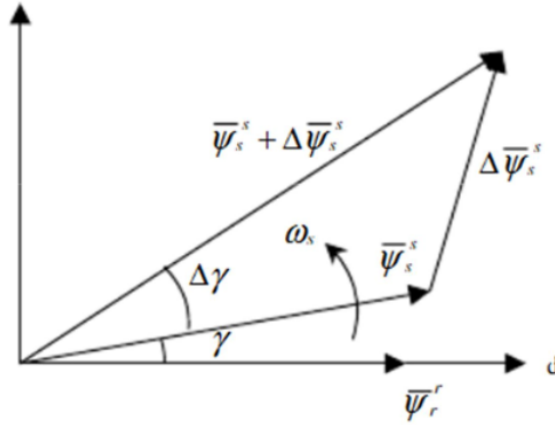


Figure 4: Stator and rotor flux space vector

Figure 4 shows the phasor for equation 9 indicating that the vectors, ψ_s , ψ_r and I for positive developed torque. If the rotor flux remains constant and the stator flux is changed incrementally by the stator voltage V_s .

The command stator flux and torque magnitudes are compared with the respective estimated values and the errors are processed through hysteresis band controller [20,21]. The torque control of the inverter fed machine is carried out by hysteresis control of magnitude of stator flux and torque that selects one of the six active and two zero inverter voltage vectors as shown in Figure 2.6. The selection is made in order to maintain the torque and flux error inside the hysteresis band in which the errors are indicated by $\Delta\psi_s$ and ΔT respectively.

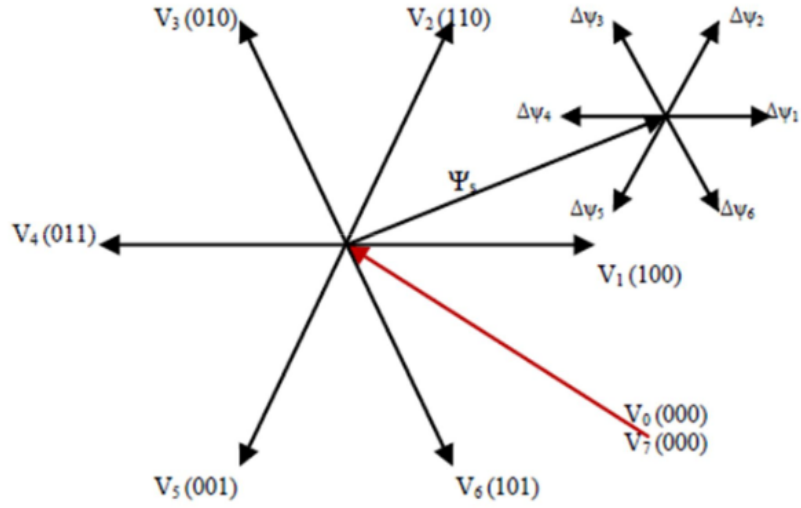


Figure 5: Inverter Voltage Vectors and corresponding stator flux variation in time

0.8.1 Flux and Torque Estimator

Flux and torque estimators are used to determine the actual value of torque and flux linkages. Into this block enters the VSI voltage vector transformed to the d-q stationary reference frame.

0.8.2 Torque and Flux Hysteresis Comparator

The estimated torque and stator flux linkage are compared with the reference torque and stator flux linkage. The error signal is processed in a comparator. If the actual flux is smaller than the reference value, the comparator output is at state 1 else it will be at state -1.

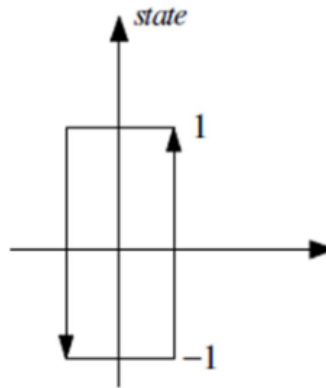


Figure 6: Flux hysteresis states

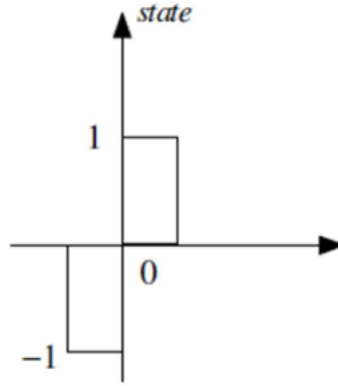


Figure 7: Torque hysteresis states

0.9 Simulink

The Simulink blocks listed below have already been discussed in previous sections:

- Flux, Torque, and Sector Estimator
- Torque Hysteresis Controller
- Lookup Table
- PI Controller

These blocks play integral roles in the overall control system, contributing to the precise operation and optimization of the motor.

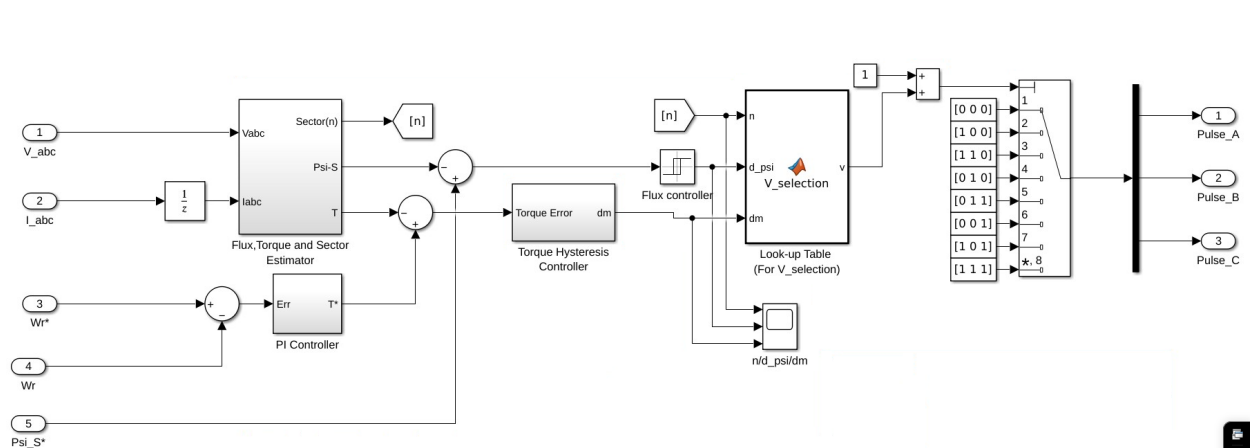


Figure 8: The Simulink model of the DTC

Long Short Term Memory

Learning long-term dependencies might be challenging for a typical RNN because it only has one hidden state that is transferred through time. A memory cell—a container that can store information for a long time—is introduced by LSTMs to solve this issue. Long-term dependencies in sequential data can be learned by LSTM networks, which makes them ideal for applications like time series forecasting, speech recognition, and language translation. Additionally, LSTMs can be utilised in conjunction with other neural network architectures for image and video analysis, such as Convolutional Neural Networks (CNNs).

Three gates—the input gate, the forget gate, and the output gate—control the memory cell. The information that is added to, subtracted from, and output from the memory cell is determined by these gates. What data is added to the memory cell is managed by the input gate. What data is erased from the memory cell is managed by the forget gate. Additionally, the output gate regulates the data that the memory cell outputs. Because of this, long-term dependencies can be learned by LSTM networks by allowing them to choose keep or reject information as it passes through the network. Figure shows the block diagram of LSTM.

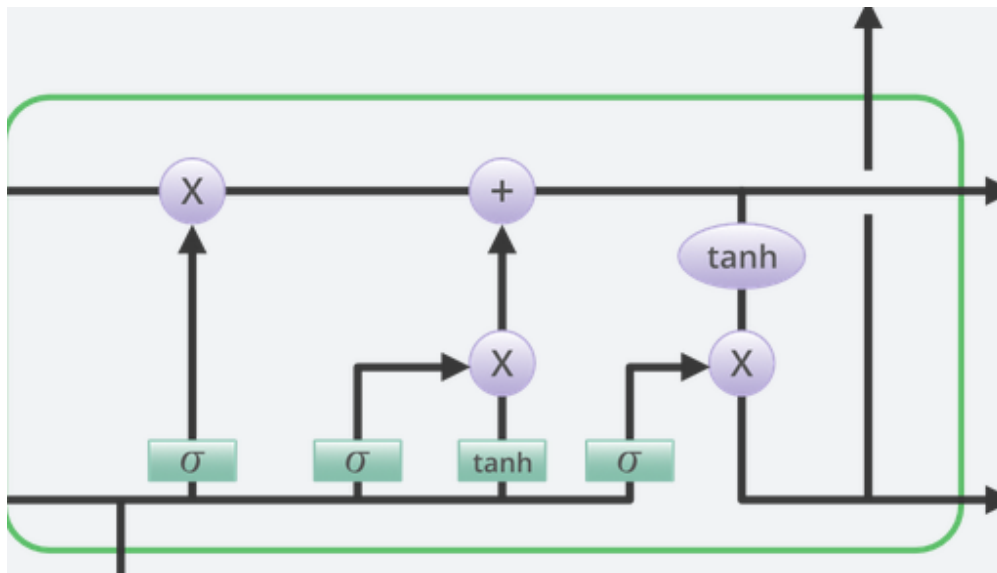


Figure 9: LSTM

0.10 Forget Gate

The forget gate eliminates data that is no longer needed in the cell state. The gate receives two inputs, x_t (the input at that specific moment) and h_{t-1} (the output of the previous cell), which are multiplied by weight matrices before bias is added. After being run through an activation function, the output is binary. When the output for a certain cell state is 0, the information is lost, and when the output is 1, it is saved for later use. The forget gate's equation is as follows:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Whereas:

- f_t represents the forget gate at time step t .
- σ is the sigmoid activation function.
- W_f is the weight matrix associated with the forget gate.
- h_{t-1} is the hidden state from the previous time step.
- x_t is the input at time step t .
- b_f is the bias term for the forget gate.

0.11 Input Gate

The addition of useful information to the cell state is done by the input gate. First, the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs h_{t-1} and x_t . Then, a vector is created using tanh function that gives an output from -1 to +1, which contains all the possible values from h_{t-1} and x_t . At last, the values of the vector and the regulated values are multiplied to obtain the useful information. The equation for the input gate is:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

- i_t represents the input gate at time step t .
- σ is the sigmoid activation function.
- W_i is the weight matrix associated with the input gate.
- h_{t-1} is the hidden state from the previous time step.
- x_t is the input at time step t .
- b_i is the bias term for the input gate.

0.12 Output gate

The task of extracting useful information from the current cell state to be presented as output is done by the output gate. First, a vector is generated by applying tanh function on the cell. Then, the information is regulated using the

sigmoid function and filter by the values to be remembered using inputs h_{t-1} and x_t . At last, the values of the vector and the regulated values are multiplied to be sent as an output and input to the next cell. The equation for the output gate is:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- o_t represents the output gate at time step t .
- σ is the sigmoid activation function.
- W_o is the weight matrix associated with the output gate.
- h_{t-1} is the hidden state from the previous time step.
- x_t is the input at time step t .
- b_o is the bias term for the output gate.

0.13 LSTM Training Results

Metric	Loss	Output Sec- tor Loss	Output Psi Loss	Output Torque Loss	Output Sec- tor Accuracy	Output Psi MSE	Output Torque MSE
Training	0.2097	0.0243	4.2555e-04	0.1850	0.9908	4.2555e-04	0.1850
Validation	0.1480	0.0232	1.9489e-04	0.1246	0.9907	1.9489e-04	0.1246

Table 1: Results of the Model

Artificial Neural Network

The conventional "Torque, Flux estimator" block has been replaced with a neural network for real-time torque and speed estimation. Figure 10 shows the simulink Model of the Neural Network implemented within the DTC model.

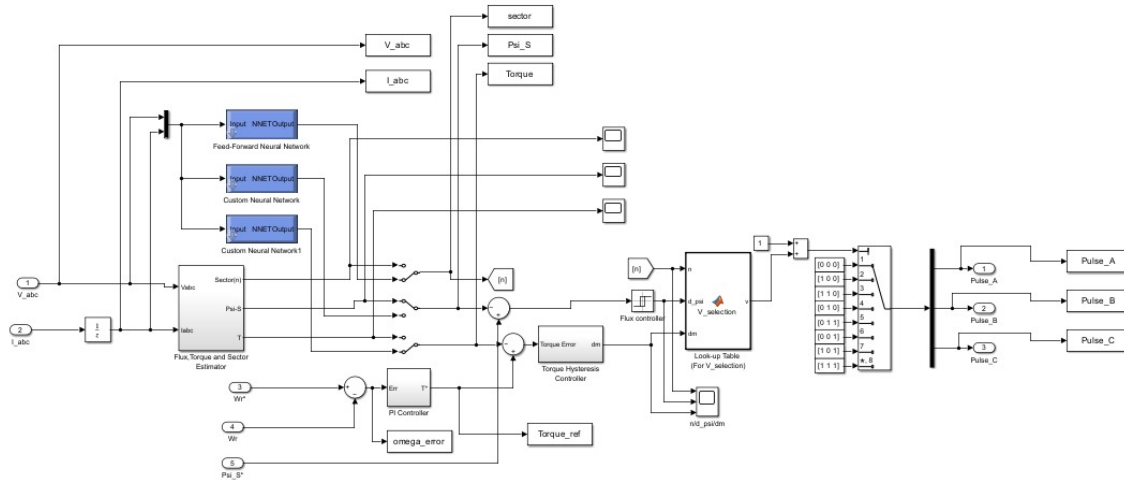


Figure 10: The Neural Network implemented within the DTC

0.14 Results

Figure 11 shows the speed control of a three phase induction motor using the typical Direct Torque Control (DTC) method.

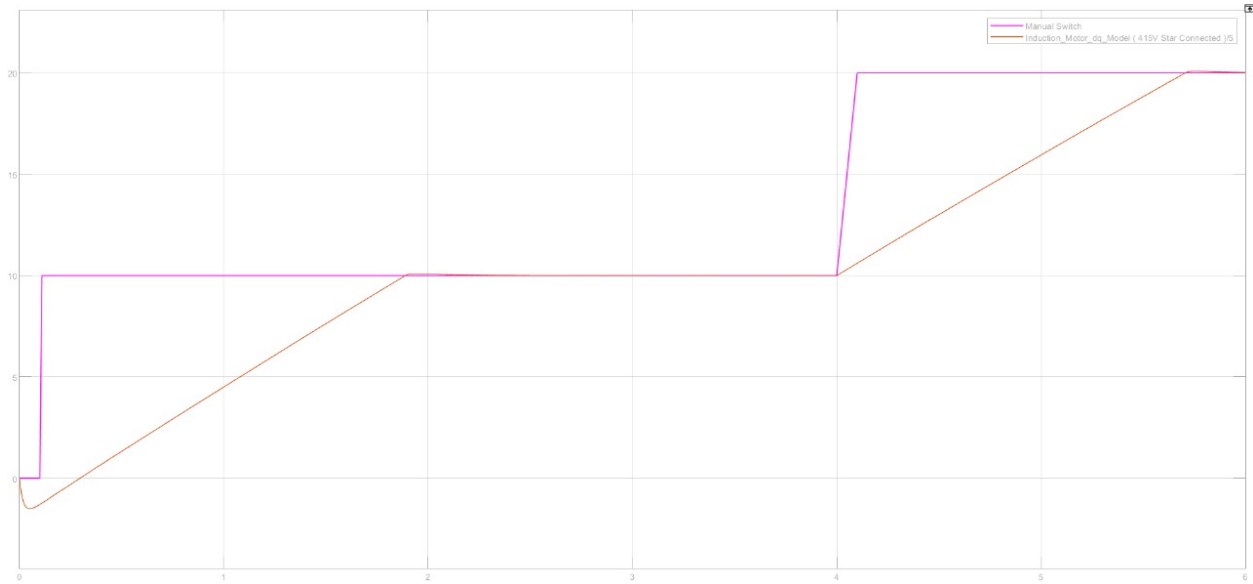


Figure 11: Speed Control using Typical DTC method

Figure 12 shows the speed control of a three phase induction motor through estimating the torque using an artificial neural network.

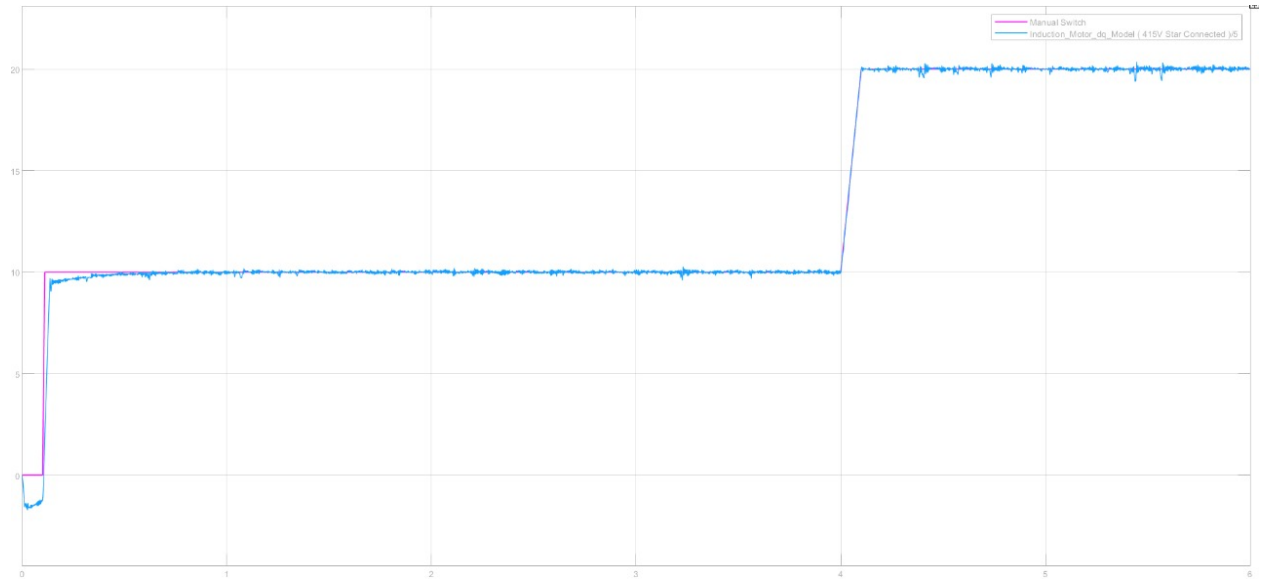


Figure 12: Speed Control through estimating the Torque using a Neural Network

Failed attempts and Iterations

Figure 13 shows the speed control of a three phase induction motor through estimating the sector and the flux using an artificial neural network.

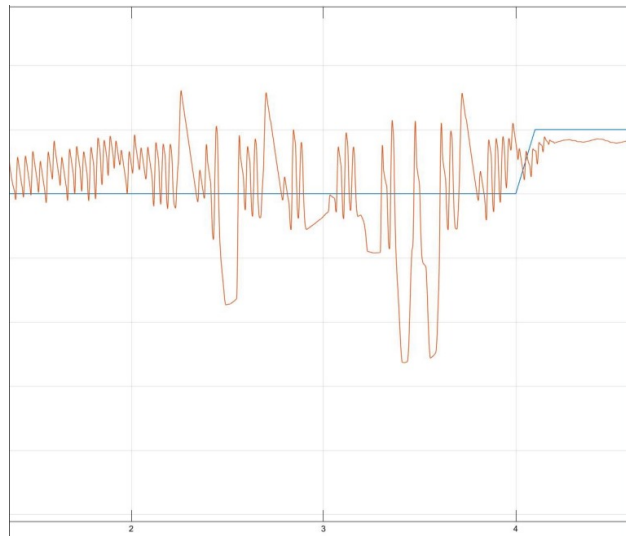


Figure 13: Speed Control through estimating the flux and sector using a Neural Network

Figure 14 shows the speed control of a three phase induction motor through estimating the Torque, sector and the flux using an artificial neural network.

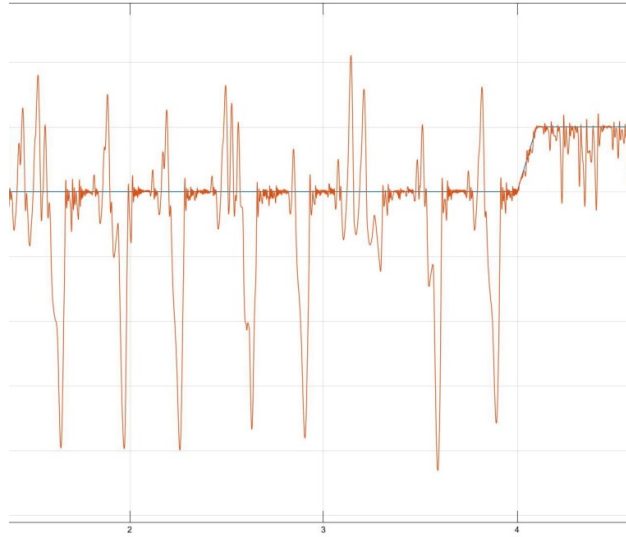


Figure 14: Speed Control through estimating the Torque, flux and sector using a Neural Network

Figure 15 shows the speed control of a three phase induction motor through estimating the Torque and sector using an artificial neural network.

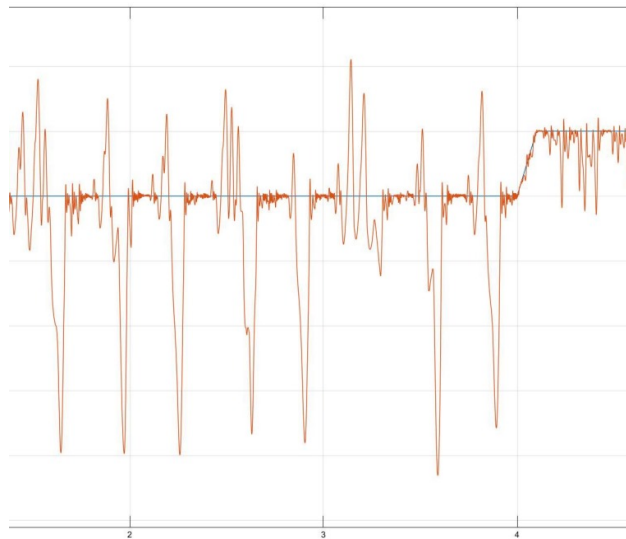


Figure 15: Speed Control through estimating the Torque and sector using a Neural Network

Conclusion and Future Work

We've found that using an artificial neural network (ANN) for estimating torque works a lot better than the traditional Direct Torque Control (DTC) method. The ANN is great at learning from data and adjusting its estimates quickly.

Although a different method called Long Short-Term Memory (LSTM) seems even better, we chose the ANN

because of time constraints. However, in the future, it would be smart to develop the LSTM further to also estimate flux along with torque. Doing this would give us a fuller picture of how the motor works and could make our control system even better.

References

- [1] Abdullah Hadi Yahya Alquhali Abdul Wali Abdul Ali. Improved internal model control technique for position control of ac servo motors. *ELEKTRIKA- Journal of Electrical Engineering*, 19:1, 2020.
- [2] Ahmed H. Eissa Ahmed S. Abd El-hamid. Position control of ac servomotor using internal model control strategy. *International Journal of Engineering Innovation Research*, 4:1, 2015.
- [3] Author B. *Title of Book*. Publisher Name, 2022.
- [4] Rached Dhaouadi and Reza Jafari. Adaptive pid neuro-controller for a nonlinear servomechanism. In *2007 IEEE International Symposium on Industrial Electronics*, pages 157–162. IEEE, 2007.
- [5] Ph.D Kashyap D Dhruve Lalithamma GA, P.S. Puttaswamy. Multi-layer neural network for servo motor control. *International Journal of Computer Applications (0975 – 8887)*, 71:32, 2013.

Appendix

Neural Network Training Code

Listing 1: Python code

```
import pandas as pd
import numpy as np
import tensorflow as tf

# Define custom callback class
class MyCallback(tf.keras.callbacks.Callback):
    def __init__(self, threshold):
        super(MyCallback, self).__init__()
        self.threshold = threshold

    def on_epoch_end(self, epoch, logs=None):
        if logs.get('loss') is not None and logs['loss'] < self.threshold:
            print(f"\nTraining_stopped_as_loss_reached_below_{self.threshold}.")
            self.model.stop_training = True

# Read data from Excel file
data = pd.read_excel('Book.xlsx')

# Extract input and output data
inputs = data['wref'].values.reshape(-1, 1) # Assuming 'Input' is the column
name for input
outputs = data[['Pulse_A', 'Pulse_B', 'Pulse_C']].values # Adjusted to select
only 3 outputs

# Define the neural network architecture
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(1,)), # Input
```

```

        layer with 128 neurons
tf.keras.layers.Dropout(0.2), # Dropout
        layer for regularization
tf.keras.layers.Dense(256, activation='relu'), # Hidden
        layer with 256 neurons
tf.keras.layers.BatchNormalization(), # Batch
        normalization layer
tf.keras.layers.Dense(256, activation='relu'), #
        Another hidden layer with 256 neurons
tf.keras.layers.Dropout(0.2), # Dropout
        layer for regularization
tf.keras.layers.Dense(256, activation='relu'), #
        Another hidden layer with 256 neurons
tf.keras.layers.BatchNormalization(), # Batch
        normalization layer
tf.keras.layers.Dense(128, activation='relu'), #
        Another hidden layer with 128 neurons
tf.keras.layers.Dropout(0.2), # Dropout
        layer for regularization
tf.keras.layers.Dense(128, activation='relu'), #
        Another hidden layer with 128 neurons
tf.keras.layers.Dense(3) # Output
        layer with 3 neurons
])

# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Prepare a callback instance
callback = MyCallback(threshold=0.0000001) # Example threshold, adjust as
needed

```

```

# Train the model with the callback
model.fit(inputs, outputs, epochs=10000, callbacks=[callback],
          validation_split=0.2, batch_size=64)

# Save the trained model
model.save('model_3outputs2.h5')

```

0.15 Testing the Neural Network Code

Listing 2: Python code

```

model = tf.keras.models.load_model('model_3outputs1.h5')

# Prepare a single input
input_data = np.array([[1000]]) # Example input, replace with your actual
    input data

# Make predictions using the input
predictions = model.predict(input_data)

print("Predictions:", predictions)

```

0.16 LSTM Code

Listing 3: Python code

```

import pandas as pd
import numpy as np
import tensorflow as tf

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping
from tensorflow.keras.layers import LSTM, Dense, Input, Dropout
from tensorflow.keras.models import Model

# Read data from Excel

```

```

data = pd.read_excel('/content/drive/MyDrive/Drives/flux-torque-sector_
    estimator.xlsx')

# Extract input and output data
inputs = data[['V_a', 'V_b', 'V_c', 'I_a', 'I_b', 'I_c']].values
output_sector = data['Sector'].values # Assuming 'Sector' is categorical
output_psi = data['Psi'].values # Assuming 'Psi' is continuous
output_torque = data['Torque'].values # Assuming 'Torque' is continuous

# Normalize input data
scaler = StandardScaler()
inputs = scaler.fit_transform(inputs)

# Convert output sector labels to integers starting from 0
output_sector_unique = np.unique(output_sector)
output_sector_mapping = {sector: idx for idx, sector in enumerate(
    output_sector_unique)}
output_sector_mapped = np.array([output_sector_mapping[sector] for sector in
    output_sector])

# Ensure output_sector_mapped has the same number of samples as inputs
output_sector_mapped = output_sector_mapped[:len(inputs)]

# Ensure output_psi and output_torque have the same number of samples as
    inputs
output_psi = output_psi[:len(inputs)]
output_torque = output_torque[:len(inputs)]

# Split data into train and validation sets
inputs_train, inputs_val, output_sector_train, output_sector_val,
    output_psi_train, output_psi_val, output_torque_train, output_torque_val =
    train_test_split(

```

```

inputs, output_sector_mapped, output_psi, output_torque, test_size=0.2,
    random_state=42)

# Reshape inputs for LSTM (add timestep dimension)
inputs_train_lstm = np.reshape(inputs_train, (inputs_train.shape[0], 1,
    inputs_train.shape[1]))
inputs_val_lstm = np.reshape(inputs_val, (inputs_val.shape[0], 1, inputs_val.
    shape[1]))

# Define model architecture with LSTM layers and named output layers
input_shape = (inputs_train.shape[1],) # Shape of input features
output_sector_classes = len(np.unique(output_sector)) # Number of classes in
    'Sector'

# Define input layers
input_layer = Input(shape=(1, input_shape[0]))

# LSTM layers with increased dropout rates
lstm_1 = LSTM(512, return_sequences=True)(input_layer)
lstm_1 = Dropout(0.5)(lstm_1) # Increased dropout rate
lstm_2 = LSTM(256, return_sequences=True)(lstm_1)
lstm_2 = Dropout(0.5)(lstm_2) # Increased dropout rate
lstm_3 = LSTM(128, return_sequences=False)(lstm_2)
lstm_3 = Dropout(0.5)(lstm_3) # Increased dropout rate

# Output layers
output_sector_layer = Dense(output_sector_classes, activation='softmax', name=
    'output_sector')(lstm_3)
output_psi_layer = Dense(1, activation='linear', name='output_psi')(lstm_3)
output_torque_layer = Dense(1, activation='linear', name='output_torque')(
    lstm_3)

```

```

# Define model
model = Model(inputs=input_layer, outputs=[output_sector_layer,
      output_psi_layer, output_torque_layer])

# Compile model with multiple loss functions for each output
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01), #
      Increased learning rate
      loss={'output_sector': 'sparse_categorical_crossentropy',
            'output_psi': 'mean_squared_error',
            'output_torque': 'mean_squared_error'},
      metrics={'output_sector': 'accuracy',
               'output_psi': 'mse',
               'output_torque': 'mse'})

# Learning rate scheduler
lr_scheduler = ReduceLROnPlateau(factor=0.5, patience=3, min_lr=0.000000001)

# Callback for early stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=50,
      restore_best_weights=True)

# Train model
history = model.fit(inputs_train_lstm,
      {'output_sector': output_sector_train,
       'output_psi': output_psi_train,
       'output_torque': output_torque_train},
      epochs=150, callbacks=[early_stopping, lr_scheduler],
      validation_data=(inputs_val_lstm,
            {'output_sector': output_sector_val,
             'output_psi': output_psi_val,
             'output_torque': output_torque_val}))

```

```
# Save trained model  
model.save('/content/drive/MyDrive/Drives/model_multi_output_lstm1.keras')
```